

用户认证

用户认证

本页提供 Kubernetes 中身份认证有关的概述，重点介绍与 Kubernetes API 有关的身
份认证。

Kubernetes 中的用户 {#users-in-kubernetes}

所有 Kubernetes 集群都有两类用户：由 Kubernetes 管理的服务账号和普通用户。

Kubernetes 假定普通用户是由一个与集群无关的服务通过以下方式之一进行管理的：

- 负责分发私钥的管理员
- 类似 Keystone 或者 Google Account 这类用户数据库
- 包含用户名和密码列表的文件

有鉴于此，**Kubernetes 并不包含用来代表普通用户账号的对象**。普通用户的信息无法通过 API 调用添加到集群中。

尽管无法通过 API 调用来添加普通用户，Kubernetes 仍然认为能够提供由集群的证书机构签名的合法证书的用户是通过身份认证的用户。基于这样的配置，Kubernetes 使用证书中的 'subject' 的通用名称（Common Name）字段（例如，"/CN=bob"）来确定用户名。接下来，基于角色访问控制（RBAC）子系统会确定用户是否有权针对某资源执行特定的操作。

与此不同，服务账号是 Kubernetes API 所管理的用户。它们被绑定到特定的名字空间，或者由 API 服务器自动创建，或者通过 API 调用创建。服务账号与一组以 Secret 保存的凭据相关，这些凭据会被挂载到 Pod 中，从而允许集群内的进程访问 Kubernetes API。

API 请求则或者与某普通用户相关联，或者与某服务账号相关联，亦或者被视作匿名请求。这意味着集群内外的每个进程在向 API 服务器发起请求时都必须通过身份认证，否则会被视作匿名用户。这里的进程可以是在某工作站上输入 kubectl 命令的操作人员，也可以是节点上的 kubelet 组件，还可以是控制面的成员。

身份认证策略 {#authentication-strategies}

Kubernetes 通过身份认证插件利用客户端证书、持有者令牌（Bearer Token）或身份认证代理（Proxy）来认证 API 请求的身份。HTTP 请求发给 API 服务器时，插件会将以下属性关联到请求本身：

- 用户名：用来辨识最终用户的字符串。常见的值可以是 kube-admin 或 jane@example.com。
- 用户 ID：用来辨识最终用户的字符串，旨在比用户名有更好的一致性和唯一性。
- 用户组：取值为一组字符串，其中各个字符串用来标明用户是某个命名的用户逻辑集合的成员。常见的值可能是 system:masters 或者 devops-team 等。
- 附加字段：一组额外的键-值映射，键是字符串，值是一组字符串；用来保存一些鉴权组件可能觉得有用的额外信息。

所有（属性）值对于身份认证系统而言都是不透明的，只有被鉴权组件解释过之后才有意义。

你可以同时启用多种身份认证方法，并且你通常会至少使用两种方法：

- 针对服务账号使用服务账号令牌
- 至少另外一种方法对用户的身份进行认证

当集群中启用了多个身份认证模块时，第一个成功地对请求完成身份认证的模块会直接做出评估决定。API 服务器并不保证身份认证模块的运行顺序。

对于所有通过身份认证的用户，system:authenticated 组都会被添加到其组列表中。

与其它身份认证协议（LDAP、SAML、Kerberos、X509 的替代模式等等）都可以通过使用一个身份认证代理或身份认证 Webhook 来实现。

X509 客户证书 {#x509-client-certs}

通过给 API 服务器传递 --client-ca-file=SOMEFILE 选项，就可以启动客户端证书身份认证。所引用的文件必须包含一个或者多个证书机构，用来验证向 API 服务器提供的客户端证书。如果提供了客户端证书并且证书被验证通过，则 subject 中的公共名称

（Common Name）就被作为请求的用户名。自 Kubernetes 1.4 开始，客户端证书还可以通过证书的 organization 字段标明用户的组成员信息。要包含用户的多个组成员信息，可以在证书中包含多个 organization 字段。

例如，使用 openssl 命令行工具生成一个证书签名请求：

```
openssl req -new -key jbeda.pem -out jbeda-csr.pem -subj "/CN=jbeda/O=app1/O=app2"
```

此命令将使用用户名 jbeda 生成一个证书签名请求（CSR），且该用户属于 "app1" 和 "app2" 两个用户组。

参阅管理证书了解如何生成客户端证书。

静态令牌文件 {#static-token-file}

当 API 服务器的命令行设置了 --token-auth-file=SOMEFILE 选项时，会从文件中读取持有者令牌。目前，令牌会长期有效，并且在不重启 API 服务器的情况下无法更改令牌列表。

令牌文件是一个 CSV 文件，包含至少 3 个列：令牌、用户名和用户的 UID。其余列被视为可选的组名。

如果要设置的组名不止一个，则对应的列必须用双引号括起来，例如：

```
token,user,uid,"group1,group2,group3"
```

在请求中放入持有者令牌 {#putting-a-bearer-token-in-a-request}

当使用持有者令牌来对某 HTTP 客户端执行身份认证时，API 服务器希望看到一个名为 Authorization 的 HTTP 头，其值格式为 Bearer。持有者令牌必须是一个可以放入 HTTP 头部值字段的字符序列，至多可使用 HTTP 的编码和引用机制。例如：如果持有者令牌为 31ada4fd-adec-460c-809a-9e56ceb75269，则其出现在 HTTP 头部时如下所示：

```
Authorization: Bearer 31ada4fd-adec-460c-809a-9e56ceb75269
```

启动引导令牌 {#bootstrap-tokens}

为了支持平滑地启动引导新的集群，Kubernetes 包含了一种动态管理的持有者令牌类型，称作 **启动引导令牌（Bootstrap Token）**。这些令牌以 Secret 的形式保存在 kube-system 名字空间中，可以被动态管理和创建。控制器管理器包含的 TokenCleaner 控制器能够在启动引导令牌过期时将其删除。

这些令牌的格式为 [a-z0-9]{6}.[a-z0-9]{16}。第一个部分是令牌的 ID；第二个部分是令牌的 Secret。你可以用如下所示的方式来在 HTTP 头部设置令牌：

```
Authorization: Bearer 781292.db7bc3a58fc5f07e
```

你必须在 API 服务器上设置 --enable-bootstrap-token-auth 标志来启用基于启动引导令牌的身份认证组件。你必须通过控制器管理器的 --controllers 标志来启用 TokenCleaner 控制器；这可以通过类似 --controllers=*,tokencleaner 这种设置来做到。如果你使用 kubeadm 来启动引导新的集群，该工具会帮你完成这些设置。

身份认证组件的认证结果为 `system:bootstrap:`，该用户属于 `system:bootstrappers` 用户组。这里的用户名和组设置都是有意设计成这样，其目的是阻止用户在启动引导集群之后继续使用这些令牌。这里的用户名和组名可以用来（并且已经被 `kubeadm` 用来）构造合适的鉴权策略，以完成启动引导新集群的工作。

请参阅启动引导令牌，以了解关于启动引导令牌身份认证组件与控制器的更深入的信息，以及如何使用 `kubeadm` 来管理这些令牌。

服务账号令牌 `{#service-account-tokens}`

服务账号（Service Account）是一种自动被启用的用户认证机制，使用经过签名的持有者令牌来验证请求。该插件可接受两个可选参数：

- `--service-account-key-file` 文件包含 PEM 编码的 x509 RSA 或 ECDSA 私钥或公钥，用于验证 ServiceAccount 令牌。这样指定的文件可以包含多个密钥，并且可以使用不同的文件多次指定此参数。若未指定，则使用 `--tls-private-key-file` 参数。
- `--service-account-lookup` 如果启用，则从 API 删除的令牌会被回收。

服务账号通常由 API 服务器自动创建并通过 ServiceAccount 准入控制器关联到集群中运行的 Pod 上。持有者令牌会挂载到 Pod 中可预知的位置，允许集群内进程与 API 服务器通信。服务账号也可以使用 Pod 规约的 `serviceAccountName` 字段显式地关联到 Pod 上。

`serviceAccountName` 通常会被忽略，因为关联关系是自动建立的。

`apiVersion: apps/v1` *# 此 apiVersion 从 Kubernetes 1.9 开始可用*

`kind: Deployment`

`metadata:`

`name: nginx-deployment`

`namespace: default`

`spec:`

`replicas: 3`

`template:`

`metadata:`

`# ...`

`spec:`

`serviceAccountName: bob-the-bot`

`containers:`

`- name: nginx`

`image: nginx:1.14.2`

在集群外部使用服务账号持有者令牌也是完全合法的，且可用来为长时间运行的、需要与 Kubernetes API 服务器通信的任务创建标识。要手动创建服务账号，可以使用 `kubectl create serviceaccount` 命令。此命令会在当前的名字空间中生成一个服务账号。

```
kubectl create serviceaccount jenkins
```

```
serviceaccount/jenkins created
```

创建相关联的令牌：

```
kubectl create token jenkins
```

```
eyJhbGciOiJSUzI1NiIsImtp...
```

所创建的令牌是一个已签名的 JWT 令牌。

已签名的 JWT 可以用作持有者令牌，并将被认证为所给的服务账号。关于如何在请求中包含令牌，请参阅前文。通常，这些令牌数据会被挂载到 Pod 中以便集群内访问 API 服务器时使用，不过也可以在集群外部使用。

服务账号被身份认证后，所确定的用户名为 `system:serviceaccount::`，并被分配到用户组 `system:serviceaccounts` 和 `system:serviceaccounts:`。

由于服务账号令牌也可以保存在 Secret API 对象中，任何能够写入这些 Secret 的用户都可以请求一个令牌，且任何能够读取这些 Secret 的用户都可以被认证为对应的服务账号。在为用户授予访问服务账号的权限以及对 Secret 的读取或写入权能时，要格外小心。

OpenID Connect (OIDC) 令牌 {#openid-connect-tokens}

OpenID Connect 是一种 OAuth2 认证方式，被某些 OAuth2 提供者支持，例如 Microsoft Entra ID、Salesforce 和 Google。协议对 OAuth2 的主要扩充体现在有一个附加字段会和访问令牌一起返回，这一字段称作 ID Token（ID 令牌）。ID 令牌是一种由服务器签名的 JWT 令牌，其中包含一些可预知的字段，例如用户的邮箱地址，

要识别用户，身份认证组件使用 OAuth2 令牌响应中的 `id_token`（而非 `access_token`）作为持有者令牌。关于如何在请求中设置令牌，可参见前文。

sequenceDiagram participant user as 用户 participant idp as 身份提供者 participant kube as kubectl participant api as API 服务器

```
user ->> idp: 1. 登录到 IdP
```

```
activate idp
```

```
idp -->> user: 2. 提供 access_token, id_token, 和 refresh_token
```

```
deactivate idp
activate user
user --> kube: 3. 调用 kubectl 并设置 --token 为 id_token 或者将令牌添加到 .kube/config
deactivate user
activate kube
kube --> api: 4. Authorization: Bearer...
deactivate kube
activate api
api --> api: 5. JWT 签名合法么?
api --> api: 6. JWT 是否已过期? (iat+exp)
api --> api: 7. 用户被授权了么?
api --> kube: 8. 已授权: 执行操作并返回结果
deactivate api
activate kube
kube --x user: 9. 返回结果
deactivate kube
```

1. 登录到你的身份服务 (Identity Provider)
2. 你的身份服务将为你提供 access_token、id_token 和 refresh_token
3. 在使用 kubectl 时, 将 id_token 设置为 --token 标志值, 或者将其直接添加到 kubeconfig 中
4. kubectl 将你的 id_token 放到一个称作 Authorization 的头部, 发送给 API 服务器
5. API 服务器将确保 JWT 的签名是有效的
6. 检查确认 id_token 尚未过期

如果使用 AuthenticationConfiguration 配置了 CEL 表达式, 则执行申领和/或用户验证。

7. 确认用户有权限执行操作
8. 鉴权成功之后, API 服务器向 kubectl 返回响应
9. kubectl 向用户提供反馈信息

由于用来验证你是谁的所有数据都在 id_token 中, Kubernetes 不需要再去联系身份服务。在一个所有请求都是无状态请求的模型中, 这一工作方式可以使得身份认证的解决方案更容易处理大规模请求。不过, 此访问也有一些挑战:

1. Kubernetes 没有提供用来触发身份认证过程的 "Web 界面"。因为不存在用来收集用户凭据的浏览器或用户接口，你必须自己先行完成对身份服务的认证过程。
2. id_token 令牌不可收回。因其属性类似于证书，其生命期一般很短（只有几分钟），所以，每隔几分钟就要获得一个新的令牌这件事可能很让人头疼。
3. 如果需要向 Kubernetes 控制面板执行身份认证，你必须使用 `kubectly proxy` 命令或者一个能够注入 id_token 的反向代理。

配置 API 服务器 {#configuring-the-api-server}

使用标志

要启用此插件，须在 API 服务器上配置以下标志：

- 参数：--oidc-issuer-url；描述：允许 API 服务器发现公开的签名密钥的服务的 URL。只接受模式为 https:// 的 URL。此值通常设置为服务的发现 URL，已更改为空路径。；示例：如果发行人的 OIDC 发现 URL 是 `https://accounts.google.com/.well-known/openid-configuration`，则此值应为 `https://accounts.provider.example`；必需？：是
- 参数：--oidc-client-id；描述：所有令牌都应发放给此客户 ID。；示例：`kubernetes`；必需？：是
- 参数：--oidc-username-claim；描述：用作用户名的 JWT 申领（JWT Claim）。默认情况下使用 `sub` 值，即最终用户的一个唯一的标识符。管理员也可以选择其他申领，例如 `email` 或者 `name`，取决于所用的身份服务。不过，除了 `email` 之外的申领都会被添加令牌发放者的 URL 作为前缀，以免与其他插件产生命名冲突。；示例：`sub`；必需？：否
- 参数：--oidc-username-prefix；描述：要添加到用户名申领之前的前缀，用来避免与现有用户名发生冲突（例如：`system: 用户`）。例如，此标志值为 `oidc:` 时将创建形如 `oidc:jane.doe` 的用户名。如果此标志未设置，且 --oidc-username-claim 标志值不是 `email`，则默认前缀为 `#`，其中 ``` 的值取自 --oidc-issuer-url 标志的设置。此标志值为 `-` 时，意味着禁止添加用户名前缀。；示例：`oidc:`；必需？：否
- 参数：--oidc-groups-claim；描述：用作用户组名的 JWT 申领。如果所指定的申领确实存在，则其值必须是一个字符串数组。；示例：`groups`；必需？：否
- 参数：--oidc-groups-prefix；描述：添加到组申领的前缀，用来避免与现有用户组名（如：`system: 组`）发生冲突。例如，此标志值为 `oidc:` 时，所得到的用户组名形如 `oidc:engineering` 和 `oidc:infra`。；示例：`oidc:`；必需？：否
- 参数：--oidc-required-claim；描述：取值为一个 `key=value` 偶对，意为 ID 令牌中必须存在的申领。如果设置了此标志，则 ID 令牌会被检查以确定是否包含取值匹

配的申领。此标志可多次重复，以指定多个申领。；示例：claim=value；必需？：否

- 参数：--oidc-ca-file；描述：指向一个 CA 证书的路径，该 CA 负责对你的身份服务的 Web 证书提供签名。默认值为宿主系统的根 CA。；示例：/etc/kubernetes/ssl/kc-ca.pem；必需？：否
- 参数：--oidc-signing-algs；描述：采纳的签名算法。默认为 "RS256"。可选值为：RS256、RS384、RS512、ES256、ES384、ES512、PS256、PS384、PS512。值由 RFC 7518 <https://tools.ietf.org/html/rfc7518#section-3.1> 定义。；示例：RS512；必需？：否

来自文件的身份认证配置 {#using-authentication-configuration}

JWT Authenticator 是一个使用 JWT 兼容令牌对 Kubernetes 用户进行身份认证的认证组件。认证组件将尝试解析原始 ID 令牌，验证它是否是由所配置的颁发者签名。用于验证签名的公钥是使用 OIDC 发现从发行者的公共端点发现的。

最小有效 JWT 负载必须包含以下申领：

```
{
  "iss": "https://example.com", // 必须与 issuer.url 匹配
  "aud": ["my-app"],           // issuer.audiences 中至少一项必须与所提供的 JWT 中的 "aud"
                                // 申领相匹配。
  "exp": 1234567890,           // 令牌过期时间为 UNIX 时间（自 1970 年 1 月 1 日 UTC 以来经过
                                // 的秒数）
  "": "user" // 这是在 claimMappings.username.claim 或
              // claimMappings.username.expression 中配置的用户名申领
}
```

配置文件方法允许你配置多个 JWT 认证组件，每个身份认证组件都有唯一的 issuer.url 和 issuer.discoveryURL。配置文件甚至允许你指定 CEL 表达式以将申领映射到用户属性，并验证申领和用户信息。当配置文件修改时，API 服务器还会自动重新加载认证组件。你可以使用

apiserver_authentication_config_controller_automatic_reload_last_timestamp_seconds 指标来监控 API 服务器上次重新加载配置的时间。

你必须使用 API 服务器上的 --authentication-config 标志指定身份认证配置的路径。如果你想使用命令行标志而不是配置文件，命令行标志仍然有效。要使用新功能（例如配置多个认证组件、为发行者设置多个受众），请切换到使用配置文件。

对于 Kubernetes v，结构化身份认证配置文件格式是 Beta 级别，并且使用该配置的机制也是 Beta 级别。如果你没有禁用集群的 StructuredAuthenticationConfiguration 特性

门控，则可以通过为 kube-apiserver 指定 `--authentication-config` 命令行参数来启用结构化身份认证。下面给出的是一个结构化身份认证配置文件的示例：

你不能同时指定 `--authentication-config` 和 `--oidc-*` 命令行参数，否则 API 服务器会报告错误，然后立即退出。如果你想切换到使用结构化身份认证配置，则必须删除 `--oidc-*` 命令行参数，并改用配置文件。

JWT 发行者配置中的 **egressSelectorType** 字段允许你指定应使用哪个出口选择器来发送与发行者相关的所有流量（发现、JWKS、分布式申领等）。此特性要求启用 `StructuredAuthenticationConfigurationEgressSelector` 特性门控。

```
---
#
# 注意：这是一个示例配置，不要将其用于你自己的集群！
#
apiVersion: apiserver.config.k8s.io/v1
kind: AuthenticationConfiguration
# 使用 JWT 兼容令牌对 Kubernetes 用户进行身份认证的认证组件列表，允许的最大认证组件数量为 64。
jwt:
- issuer:
    # url 在所有认证组件中必须是唯一的。
    # url 不得与 --service-account-issuer 中配置的颁发者冲突。
    url: https://example.com # 与 --oidc-issuer-url 一致。
    # discoveryURL（如果指定）将覆盖用于获取发现信息的 URL，而不是使用 “{url}/.well-known/openid-configuration”。
    # 系统会使用所给的配置值，因此如果需要， “/.well-known/openid-configuration” 必须包含在 discoveryURL 中。
    #
    # 取回的发现信息中的 “issuer” 字段必须与 AuthenticationConfiguration 中的
    # “issuer.url” 字段匹配，并被用于验证所呈现的 JWT 中的 “iss” 申领。
    # 这适用于众所周知的端点和 jwks 端点托管在与颁发者不同的位置（例如集群本地）的场景。
    # discoveryURL 必须与 url 不同（如果指定），并且在所有认证组件中必须是唯一的。
    discoveryURL: https://discovery.example.com/.well-known/openid-configuration
    # PEM 编码的 CA 证书用于在获取发现信息时验证连接。
    # 如果未设置，将使用系统验证程序。
    # 与 --oidc-ca-file 标志引用的文件内容的值相同。
  certificateAuthority:
    # audiences 是 JWT 必须发布给的一组可接受的受众。
    # 至少其中一项必须与所提供的 JWT 中的 “aud” 申领相匹配。
  audiences:
```

- my-app # 与 --oidc-client-id 一致。
- my-other-app
- # 当指定多个受众时，需要将此字段设置为 “MatchAny”。
- audienceMatchPolicy: MatchAny
- # egressSelectorType 是一个指示符，用于指定应使用哪个出口选择器发送与此发行者相关的所有流量
- # (发现、JWKS、分布式申领等)。
- # 如果未指定，则不使用自定义拨号器。
- # 当指定时，有效选项为 "controlplane" 和 "cluster"。这些对应于
- # --egress-selector-config-file 中的相关值。
- # - controlplane: 用于打算发往控制平面的流量。
- # - cluster: 用于打算发往由 Kubernetes 管理的系统的流量。
- egressSelectorType:
- # 用于验证令牌申领以对用户进行身份认证的规则。
- claimValidationRules:
- # 与 --oidc-required-claim key=value 一致
- claim: hd
- requiredValue: example.com
- # 你可以使用表达式来验证申领，而不是仅仅靠 claim 和 requiredValue 来执行检查。
- # expression 是一个计算结果为布尔值的 CEL 表达式。
- # 所有表达式的计算结果必须为 true 才能使验证成功。
- expression: 'claims.hd == "example.com"'
- # message 用来定制验证失败时在 API 服务器日志中看到的错误消息。
- message: the hd claim must be set to example.com
- expression: 'claims.exp - claims.nbf`': 一个动态的头部字段，用来设置与用户相关的附加字段。

此字段可选；要求 "Impersonate-User" 被设置。为了能够以一致的形式保留，
 ``部分必须是小写字符，
 如果有任何字符不是合法的 HTTP 头部标签字符，
 则必须是 UTF-8 字符，且转换为百分号编码。

* `Impersonate-Uid`：一个唯一标识符，用来表示所伪装的用户。此头部可选。
 如果设置，则要求 "Impersonate-User" 也存在。Kubernetes 对此字符串没有格式要求。

在 1.11.3 版本之前（以及 1.10.7、1.9.11），``只能包含合法的 HTTP 标签字符。

`Impersonate-Uid` 仅在 1.22.0 及更高版本中可用。

伪装带有用户组的用户时，所使用的伪装头部字段示例：

```http

```
Impersonate-User: jane.doe@example.com
Impersonate-Group: developers
Impersonate-Group: admins
```

伪装带有 UID 和附加字段的用户时，所使用的伪装头部字段示例：

```
Impersonate-User: jane.doe@example.com
Impersonate-Extra-dn: cn=jane,ou=engineers,dc=example,dc=com
Impersonate-Extra-acme.com%2Fproject: some-project
Impersonate-Extra-scopes: view
Impersonate-Extra-scopes: development
Impersonate-Uid: 06f6ce97-e2c5-4ab8-7ba5-7654dd08d52b
```

在使用 kubectl 时，可以使用 --as 标志来配置 Impersonate-User 头部字段值，使用 --as-group 标志配置 Impersonate-Group 头部字段值。

```
kubectl drain mynode
```

```
Error from server (Forbidden): User "clark" cannot get nodes at the cluster scope. (get
nodes mynode)
```

设置 --as 和 --as-group 标志：

```
kubectl drain mynode --as=superman --as-group=system:masters
```

```
node/mynode cordoned
node/mynode drained
```

kubectl 不能对附加字段或 UID 执行伪装。

若要伪装成某个用户、某个组、用户标识符（UID））或者设置附加字段，执行伪装操作的用户必须具有对所伪装的类别（user、group、uid 等）执行 impersonate 动词操作的能力。对于启用了 RBAC 鉴权插件的集群，下面的 ClusterRole 封装了设置用户和组伪装字段所需的规则：

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
 name: impersonator
rules:
- apiGroups: [""]
 resources: ["users", "groups", "serviceaccounts"]
 verbs: ["impersonate"]
```

为了执行伪装，附加字段和所伪装的 UID 都位于 "authorization.k8s.io" apiGroup 中。附加字段会被作为 userextras 资源的子资源来执行权限评估。如果要允许用户为附加字段 “scopes” 和 UID 设置伪装头部，该用户需要被授予以下角色：

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
 name: scopes-and-uid-impersonator
rules:
 # 可以设置 "Impersonate-Extra-scopes" 和 "Impersonate-Uid" 头部
 - apiGroups: ["authentication.k8s.io"]
 resources: ["userextras/scopes", "uids"]
 verbs: ["impersonate"]
```

你也可以通过约束资源可能对应的 resourceNames 限制伪装头部的取值：

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
 name: limited-impersonator
rules:
 # 可以伪装成用户 "jane.doe@example.com"
 - apiGroups: [""]
 resources: ["users"]
 verbs: ["impersonate"]
 resourceNames: ["jane.doe@example.com"]

 # 可以伪装成用户组 "developers" 和 "admins"
 - apiGroups: [""]
 resources: ["groups"]
 verbs: ["impersonate"]
 resourceNames: ["developers", "admins"]

 # 可以将附加字段 "scopes" 伪装成 "view" 和 "development"
 - apiGroups: ["authentication.k8s.io"]
 resources: ["userextras/scopes"]
 verbs: ["impersonate"]
 resourceNames: ["view", "development"]

 # 可以伪装 UID "06f6ce97-e2c5-4ab8-7ba5-7654dd08d52b"
 - apiGroups: ["authentication.k8s.io"]
```

```
resources: ["uids"]
verbs: ["impersonate"]
resourceNames: ["06f6ce97-e2c5-4ab8-7ba5-7654dd08d52b"]
```

基于伪装成一个用户或用户组的能力，你可以执行任何操作，好像你就是那个用户或用户组一样。出于这一原因，伪装操作是不受名字空间约束的。如果你希望允许使用 Kubernetes RBAC 来执行身份伪装，就需要使用 ClusterRole 和 ClusterRoleBinding，而不是 Role 或 RoleBinding。

## client-go 凭据插件 {#client-go-credential-plugins}

k8s.io/client-go 及使用它的工具（如 kubectl 和 kubelet）可以执行某个外部命令来获得用户的凭据信息。

这一特性的目的是便于客户端与 k8s.io/client-go 并不支持的身份认证协议（LDAP、Kerberos、OAuth2、SAML 等）继承。插件实现特定于协议的逻辑，之后返回不透明的凭据以供使用。几乎所有的凭据插件使用场景中都需要在服务器端存在一个支持 Webhook 令牌身份认证组件的模块，负责解析客户端插件所生成的凭据格式。

早期版本的 kubectl 内置了对 AKS 和 GKE 的认证支持，但这一功能已不再存在。

## 示例应用场景 {#example-use-case}

在一个假想的应用场景中，某组织运行这一个外部的服务，能够将特定用户的已签名的令牌转换成 LDAP 凭据。此服务还能够对 Webhook 令牌身份认证组件的请求做出响应以验证所提供的令牌。用户需要在工作站上安装一个凭据插件。

要对 API 服务器认证身份时：

- 用户发出 kubectl 命令。
- 凭据插件提示用户输入 LDAP 凭据，并与外部服务交互，获得令牌。
- 凭据插件将令牌返回给 client-go，后者将其用作持有者令牌提交给 API 服务器。
- API 服务器使用 Webhook 令牌身份认证组件向外部服务发出 TokenReview 请求。
- 外部服务检查令牌上的签名，返回用户的用户名和用户组信息。

## 配置 {#configuration}

凭据插件通过 kubectl 配置文件 来作为 user 字段的一部分设置。

```
apiVersion: v1
kind: Config
users:
```

```

- name: my-user
 user:
 exec:
 # 要执行的命令。必需。
 command: "example-client-go-exec-plugin"

 # 解析 ExecCredentials 资源时使用的 API 版本。必需。
 # 插件返回的 API 版本必需与这里列出的版本匹配。
 #
 # 要与支持多个版本的工具（如 client.authentication.k8s.io/v1beta1）集成，
 # 可以设置一个环境变量或者向工具传递一个参数标明 exec 插件所期望的版本，
 # 或者从 KUBERNETES_EXEC_INFO 环境变量的 ExecCredential 对象中读取版本信息。
 apiVersion: "client.authentication.k8s.io/v1"

 # 执行此插件时要设置的环境变量。可选字段。
 env:
 - name: "FOO"
 value: "bar"

 # 执行插件时要传递的参数。可选字段。
 args:
 - "arg1"
 - "arg2"

 # 当可执行文件不存在时显示给用户的文本。可选字段。
 installHint: |
 需要 example-client-go-exec-plugin 来在当前集群上执行身份认证。可以通过以下命令安
装：

 MacOS: brew install example-client-go-exec-plugin

 Ubuntu: apt-get install example-client-go-exec-plugin

 Fedora: dnf install example-client-go-exec-plugin

 ...

 # 是否使用 KUBERNETES_EXEC_INFO 环境变量的一部分向这个 exec 插件
 # 提供集群信息（可能包含非常大的 CA 数据）

```



```
provideClusterInfo: true

Exec 插件与标准输入 I/O 数据流之间的协议。如果协议无法满足，
则插件无法运行并会返回错误信息。合法的值包括 "Never"（Exec 插件从不使用标准输入），
"IfAvailable"（Exec 插件希望在可以的情况下使用标准输入），
或者 "Always"（Exec 插件需要使用标准输入才能工作）。必需字段。
interactiveMode: Never
clusters:
- name: my-cluster
 cluster:
 server: "https://172.17.4.100:6443"
 certificate-authority: "/etc/kubernetes/ca.pem"
 extensions:
 - name: client.authentication.k8s.io/exec # 为每个集群 exec 配置保留的扩展名
 extension:
 arbitrary: config
 this: 在设置 provideClusterInfo 时可通过环境变量 KUBERNETES_EXEC_INFO 指定
 you: ["can", "put", "anything", "here"]
 contexts:
 - name: my-cluster
 context:
 cluster: my-cluster
 user: my-user
 current-context: my-cluster

apiVersion: v1
kind: Config
users:
- name: my-user
 user:
 exec:
 # 要执行的命令。必需。
 command: "example-client-go-exec-plugin"

解析 ExecCredentials 资源时使用的 API 版本。必需。
插件返回的 API 版本必需与这里列出的版本匹配。
#
要与支持多个版本的工具（如 client.authentication.k8s.io/v1）集成，
可以设置一个环境变量或者向工具传递一个参数标明 exec 插件所期望的版本，
```

*# 或者从 KUBERNETES\_EXEC\_INFO 环境变量的 ExecCredential 对象中读取版本信息。*  
**apiVersion:** "client.authentication.k8s.io/v1beta1"

*# 执行此插件时要设置的环境变量。可选字段。*

**env:**

- **name:** "FOO"  
  **value:** "bar"

*# 执行插件时要传递的参数。可选字段。*

**args:**

- "arg1"  
- "arg2"

*# 当可执行文件不存在时显示给用户的文本。可选字段。*

**installHint:** |

需要 example-client-go-exec-plugin 来在当前集群上执行身份认证。可以通过以下命令安装：

MacOS: brew install example-client-go-exec-plugin

Ubuntu: apt-get install example-client-go-exec-plugin

Fedora: dnf install example-client-go-exec-plugin

...

*# 是否使用 KUBERNETES\_EXEC\_INFO 环境变量的一部分向这个 exec 插件*

*# 提供集群信息（可能包含非常大的 CA 数据）*

**provideClusterInfo:** true

*# Exec 插件与标准输入 I/O 数据流之间的协议。如果协议无法满足，*

*# 则插件无法运行并会返回错误信息。合法的值包括 "Never"（Exec 插件从不使用标准输入），*

*# "IfAvailable"（Exec 插件希望在可以的情况下使用标准输入），*

*# 或者 "Always"（Exec 插件需要使用标准输入才能工作）。可选字段。*

*# 默认值为 "IfAvailable"。*

**interactiveMode:** Never

**clusters:**

- **name:** my-cluster

```

cluster:
 server: "https://172.17.4.100:6443"
 certificate-authority: "/etc/kubernetes/ca.pem"
 extensions:
 - name: client.authentication.k8s.io/exec # 为每个集群 exec 配置保留的扩展名
 extension:
 arbitrary: config
 this: 在设置 provideClusterInfo 时可通过环境变量 KUBERNETES_EXEC_INFO 指定
 you: ["can", "put", "anything", "here"]
contexts:
- name: my-cluster
 context:
 cluster: my-cluster
 user: my-user
current-context: my-cluster

```

解析相对命令路径时，kubectl 将其视为与配置文件比较而言的相对路径。如果 KUBECONFIG 被设置为 /home/jane/kubeconfig，而 exec 命令为 ./bin/example-client-go-exec-plugin，则要执行的可执行文件为 /home/jane/bin/example-client-go-exec-plugin。

```

- name: my-user
 user:
 exec:
 # 对 kubeconfig 目录而言的相对路径
 command: "./bin/example-client-go-exec-plugin"
 apiVersion: "client.authentication.k8s.io/v1"
 interactiveMode: Never

```

## 输出和输出格式 {#input-and-output-formats}

所执行的命令会在 stdout 打印 ExecCredential 对象。k8s.io/client-go 使用 status 中返回的凭据信息向 Kubernetes API 服务器执行身份认证。所执行的命令会通过环境变量 KUBERNETES\_EXEC\_INFO 收到一个 ExecCredential 对象作为其输入。此输入中包含类似于所返回的 ExecCredential 对象的预期 API 版本，以及是否插件可以使用 stdin 与用户交互这类信息。

在交互式会话（即，某终端）中运行时，stdin 是直接暴露给插件使用的。插件应该使用来自 KUBERNETES\_EXEC\_INFO 环境变量的 ExecCredential 输入对象中的 spec.interactive 字段来确定是否提供了 stdin。插件的 stdin 需求（即，为了能够让插件成功运行，是否 stdin 是可选的、必须提供的或者从不会被使用的）是通过 kubeconfig 中的

user.exec.interactiveMode 来申领的（参见下面的表格了解合法值）。字段 user.exec.interactiveMode 在 client.authentication.k8s.io/v1beta1 中是可选的，在 client.authentication.k8s.io/v1 中是必需的。

- interactiveMode 取值：Never；含义：此 exec 插件从不需要使用标准输入，因此如论是否有标准输入提供给用户输入，该 exec 插件都能运行。
- interactiveMode 取值：IfAvailable；含义：此 exec 插件希望在标准输入可用的情况下使用标准输入，但在标准输入不存在时也可运行。因此，无论是否存在给用户提供输入的标准输入，此 exec 插件都会运行。如果存在供用户输入的标准输入，则该标准输入会被提供给 exec 插件。
- interactiveMode 取值：Always；含义：此 exec 插件需要标准输入才能正常运行，因此只有存在供用户输入的标准输入时，此 exec 插件才会运行。如果不存在供用户输入的标准输入，则 exec 插件无法运行，并且 exec 插件的返回者会因此返回错误信息。

要使用所有者令牌凭据，此插件将在 ExecCredential 的状态中返回一个令牌：

```
{
 "apiVersion": "client.authentication.k8s.io/v1",
 "kind": "ExecCredential",
 "status": {
 "token": "my-bearer-token"
 }
}

{
 "apiVersion": "client.authentication.k8s.io/v1beta1",
 "kind": "ExecCredential",
 "status": {
 "token": "my-bearer-token"
 }
}
```

另一种方案是，返回 PEM 编码的客户端证书和密钥，以便执行 TLS 客户端身份认证。如果插件在后续调用中返回了不同的证书或密钥，k8s.io/client-go 会终止其与服务器的连接，从而强制执行新的 TLS 握手过程。

如果指定了这种方式，则 clientKeyData 和 clientCertificateData 字段都必须存在。

clientCertificateData 字段可能包含一些要发送给服务器的中间证书（Intermediate Certificates）。

```
{
 "apiVersion": "client.authentication.k8s.io/v1",
 "kind": "ExecCredential",
 "status": {
 "clientCertificateData": "-----BEGIN CERTIFICATE-----\n...\n-----END CERTIFICATE-----",
 "clientKeyData": "-----BEGIN RSA PRIVATE KEY-----\n...\n-----END RSA PRIVATE KEY-----"
 }
}

{
 "apiVersion": "client.authentication.k8s.io/v1beta1",
 "kind": "ExecCredential",
 "status": {
 "clientCertificateData": "-----BEGIN CERTIFICATE-----\n...\n-----END CERTIFICATE-----",
 "clientKeyData": "-----BEGIN RSA PRIVATE KEY-----\n...\n-----END RSA PRIVATE KEY-----"
 }
}
```

作为一种可选方案，响应中还可以包含以 RFC 3339 时间戳格式给出的证书到期时间。证书到期时间的有无会有如下影响：

- 如果响应中包含了到期时间，持有者令牌和 TLS 凭据会被缓存，直到期限到来、或者服务器返回 401 HTTP 状态码，或者进程退出。
- 如果未指定到期时间，则持有者令牌和 TLS 凭据会被缓存，直到服务器返回 401 HTTP 状态码或者进程退出。

```
{
 "apiVersion": "client.authentication.k8s.io/v1",
 "kind": "ExecCredential",
 "status": {
 "token": "my-bearer-token",
 "expirationTimestamp": "2018-03-05T17:30:20-08:00"
 }
}

{
 "apiVersion": "client.authentication.k8s.io/v1beta1",
 "kind": "ExecCredential",
 "status": {
 "token": "my-bearer-token",
 "expirationTimestamp": "2018-03-05T17:30:20-08:00"
 }
}
```

```
}
}
```

为了让 exec 插件能够获得特定与集群的信息，可以在 kubeconfig 中的 user.exec 设置 provideClusterInfo。这一特定于集群的信息就会通过 KUBERNETES\_EXEC\_INFO 环境变量传递给插件。此环境变量中的信息可以用来执行特定于集群的凭据获取逻辑。下面的 ExecCredential 清单描述的是一个示例集群信息。

```
{
 "apiVersion": "client.authentication.k8s.io/v1",
 "kind": "ExecCredential",
 "spec": {
 "cluster": {
 "server": "https://172.17.4.100:6443",
 "certificate-authority-data": "LS0t...",
 "config": {
 "arbitrary": "config",
 "this": "可以在设置 provideClusterInfo 时通过 KUBERNETES_EXEC_INFO 环境变量提供",
 "you": ["can", "put", "anything", "here"]
 }
 },
 "interactive": true
 }
}

{
 "apiVersion": "client.authentication.k8s.io/v1beta1",
 "kind": "ExecCredential",
 "spec": {
 "cluster": {
 "server": "https://172.17.4.100:6443",
 "certificate-authority-data": "LS0t...",
 "config": {
 "arbitrary": "config",
 "this": "可以在设置 provideClusterInfo 时通过 KUBERNETES_EXEC_INFO 环境变量提供",
 "you": ["can", "put", "anything", "here"]
 }
 },
 "interactive": true
 }
}
```

## 为客户端提供的对身份认证信息的 API 访问 {#self-subject-review}

如果集群启用了此 API，你可以使用 SelfSubjectReview API 来了解 Kubernetes 集群如何映射你的身份认证信息从而将你识别为某客户端。无论你是作为用户（通常代表一个真实的人）还是作为 ServiceAccount 进行身份认证，这一 API 都可以使用。

SelfSubjectReview 对象没有任何可配置的字段。Kubernetes API 服务器收到请求后，将使用用户属性填充 status 字段并将其返回给用户。

请求示例（主体将是 SelfSubjectReview）：

POST /apis/authentication.k8s.io/v1/selfsubjectreviews

```
{
 "apiVersion": "authentication.k8s.io/v1",
 "kind": "SelfSubjectReview"
}
```

响应示例：

```
{
 "apiVersion": "authentication.k8s.io/v1",
 "kind": "SelfSubjectReview",
 "status": {
 "userInfo": {
 "name": "jane.doe",
 "uid": "b6c7cfd4-f166-11ec-8ea0-0242ac120002",
 "groups": [
 "viewers",
 "editors",
 "system:authenticated"
],
 "extra": {
 "provider_id": ["token.company.example"]
 }
 }
 }
}
```

为了方便，Kubernetes 提供了 kubectl auth whoami 命令。执行此命令将产生以下输出（但将显示不同的用户属性）：

- 简单的输出示例



| ATTRIBUTE | VALUE                  |
|-----------|------------------------|
| Username  | jane.doe               |
| Groups    | [system:authenticated] |

- 包括额外属性的复杂示例

| ATTRIBUTE       | VALUE                                    |
|-----------------|------------------------------------------|
| Username        | jane.doe                                 |
| UID             | b79dbf30-0c6a-11ed-861d-0242ac120002     |
| Groups          | [students teachers system:authenticated] |
| Extra: skills   | [reading learning]                       |
| Extra: subjects | [math sports]                            |

通过提供 output 标志，也可以打印结果的 JSON 或 YAML 表现形式：

```
{
 "apiVersion": "authentication.k8s.io/v1alpha1",
 "kind": "SelfSubjectReview",
 "status": {
 "userInfo": {
 "username": "jane.doe",
 "uid": "b79dbf30-0c6a-11ed-861d-0242ac120002",
 "groups": [
 "students",
 "teachers",
 "system:authenticated"
],
 "extra": {
 "skills": [
 "reading",
 "learning"
],
 "subjects": [
 "math",
 "sports"
]
 }
 }
 }
}
```

```
apiVersion: authentication.k8s.io/v1
kind: SelfSubjectReview
status:
 userInfo:
 username: jane.doe
 uid: b79dbf30-0c6a-11ed-861d-0242ac120002
 groups:
 - students
 - teachers
 - system:authenticated
 extra:
 skills:
 - reading
 - learning
 subjects:
 - math
 - sports
```

在 Kubernetes 集群中使用复杂的身份认证流程时，例如如果你使用 Webhook 令牌身份认证或 身份认证代理时，此特性极其有用。

Kubernetes API 服务器在所有身份认证机制（包括伪装），被应用后填充 userInfo，如果你或某个身份认证代理使用伪装进行 SelfSubjectReview，你会看到被伪装用户的用户详情和属性。

默认情况下，所有经过身份认证的用户都可以在 APISelfSubjectReview 特性被启用时创建 SelfSubjectReview 对象。这是 system:basic-user 集群角色允许的操作。

你只能在以下情况下进行 SelfSubjectReview 请求：

- 集群启用了 APISelfSubjectReview 特性门控（Kubernetes 不需要，但较旧的 Kubernetes 版本可能没有此特性门控，或者默认为关闭状态）。
  - （如果你运行的 Kubernetes 版本早于 v1.28 版本）集群的 API 服务器包含 authentication.k8s.io/v1alpha1 或 authentication.k8s.io/v1beta1 API 组。
  - 集群的 API 服务器已启用 authentication.k8s.io/v1alpha1 或者 authentication.k8s.io/v1beta1 。
- 
- 要了解为用户颁发证书的有关信息，阅读使用 CertificateSigningRequest 为 Kubernetes API 客户端颁发证书。

- 阅读客户端认证参考文档（v1）。
- 阅读客户端认证参考文档（v1beta1）。